Rockset Converged Index Architecture

Unifying Row, Column & Inverted Indexes in RocksDB Key-Value Space

3-in-1
ROW + COLUMN + INVERTED INDEX ON ALL FIELDS

0 Manual Indexes
AUTOMATIC SEMI-STRUCTURED JSON SHREDDING

1. The Indexing Trilemma in Database Systems

Traditional data systems force software engineers into a structural compromise:

- **Row Stores (Postgres/MySQL):** Fast primary-key point lookups, but reading whole rows across disk blocks creates massive IO bottlenecks during analytical aggregations.
- **Column Stores (ClickHouse/Snowflake):** Ideal for vectorized scans and aggregations, but slow at selective point lookups and entity reconstructions.
- **Inverted Search Indexes (Elasticsearch):** Highly selective predicate filters and token searches, but incapable of SQL joins and inefficient for numeric math.

2. The Converged Index™ Paradigm

Rockset resolves this trilemma by building **all three index representations simultaneously** for every field in an incoming document, mapped directly into sorted RocksDB byte keys:

Index Component	RocksDB Key Layout	Primary Workload Served
Inverted Index	`[ColID] : [Field] : [Type] : [Value] : [DocID] → ∅`	High-selectivity predicate filters (`WHERE status = 'PAID'`).
Columnar Index	`[ColID] : [Field] : [ChunkID] → [Array of Values]`	SIMD vectorized scans & aggregations (`SUM(price)`).
Row Store	`[ColID] : [DocID] : [Field] → [FieldValue]`	Point lookups & whole-document JSON reconstruction.

3. Smart Schema: Deep JSON Shredding

Incoming semi-structured JSON is parsed without up-front schema declarations. Nested paths (e.g., `user.address.geo.lat`) are extracted into distinct path-value tuples. Polymorphic and mixed types (e.g., string vs numeric integer) coexist cleanly under distinct `[Type]` prefix markers, enabling schema-free operation with full ANSI SQL queryability.

4. Physical RocksDB Byte Key Encodings

RocksDB sorts keys lexicographically. Rockset encodes keys to optimize range seek performance:

1. Inverted Search Key Encoding

`0x01 | Collection_UUID (16B) | Field_ID (4B) | Type_Tag (1B) | Normalized_Value | Doc_ID (16B)`

A prefix seek on `0x01 | UUID | Field_ID | Type | Value` traverses only the exact matching document IDs via sequential RocksDB iterators.

2. Columnar Chunk Key Encoding

`0x02 | Collection_UUID (16B) | Field_ID (4B) | Chunk_Number (8B)` → `[Zstd Compressed Vector]`

Contiguous value chunks (e.g., 64-256 elements) allow SIMD vector registers (AVX2/AVX-512) to compute aggregations at memory bus saturation speed.

5. Query Optimizer Synergy Across Index Types

Rockset's cost-based query optimizer executes queries by combining the strengths of multiple index structures within a single physical plan:

```
-- Query combining high selectivity with analytical math
SELECT country, AVG(order_total)
FROM orders
WHERE status = 'DELIVERED' AND order_date > '2026-01-01'
GROUP BY country;
```

Execution Plan Walkthrough

Phase 1 (Inverted Index): Evaluates `status = 'DELIVERED'` and date range. Produces a compressed Roaring Bitmap of qualifying document IDs in microseconds.

Phase 2 (Columnar Index): Evaluates `AVG(order_total)` and `country` by scanning exclusively the qualifying rows in the Columnar chunks. Non-queried fields (e.g. shipping addresses) are never paged into RAM.

6. Storage Compression & Footprint Economics

Building three index representations simultaneously raises concerns regarding storage inflation. Rockset neutralizes storage bloat through specialized compression algorithms:

- **Dictionary Encoding:** Low-cardinality strings are replaced with 1-byte integer tokens.
- **Frame-of-Reference (FoR) Bit-Packing:** Consecutive timestamps and numeric IDs are stored as compressed integer deltas.
- **Prefix Compression & Zstandard (zstd):** Inverted index keys share identical collection and field prefixes, allowing RocksDB block prefix compression and zstd to compress keys by up to 80%.

7. Storage Footprint vs. Alternative Engines

Storage Metric	Elasticsearch (Source + DocValues)	Snowflake / ClickHouse	Rockset Converged Index
Storage Multiplier	2.0× – 3.0× of raw JSON	0.5× – 1.0× (Columnar only)	**1.5× – 2.0× of raw JSON**
Point Lookup Speed	10 – 30 ms (Doc ID fetch)	150 – 500 ms (Scan & stitch)	**1 – 3 ms (Row Key seek)**
Aggregation Speed	Slow / Memory Heavy	Fast (Vectorized)	**Fast (SIMD Columnar)**
Index Tuning Burden	Manual mapping definitions	Sorting keys / partition keys	**Zero manual tuning required**

8. Architectural Hardening Best Practices

1. Write Amplification Tamed by LSM Sequential Appends

While one document write creates 15–20 RocksDB key-value entries, the LSM-tree converts them into sequential in-memory MemTable appends, sustaining 100k+ writes/sec without write-lock stalls.

2. Selective Index Exclusions for Massive Blobs

For extremely wide documents with multi-megabyte raw payload fields, configure selective indexing to omit heavy payload text from the inverted index while preserving row retrieval.